CMPE 125 Spring 2026
Tuesday 1:30-4:15 PM
Lab #7: ALU
Olivia Chen
ID: 017098263

DESIGN & CALCULATIONS

Background:

The ALU's job is to execute arithmetic operations through combinations of the 6 operations it directly supports: AND, OR, add, sub, slt (set less than), and NOR. It also computes the three flag bits: Z (zero flag), V (overflow flag), and C (carry flag). The combinational circuit in the ALU calculates a 32-bit output based on two 32-bit inputs and a 4-bit input specifying the ALU operation to perform: (For the purpose of this lab, we are only doing AND, OR, add/sub, and a sign bit output)

Function	ALU control	Semantics
AND	0000	F = A & B (bitwise AND); Z update, V,C=0
OR	0001	F = A | B (bitwise OR); Z update, V,C=0
add	0010	F = A + B; Z,V,C update
sub	0110	F = A - B; Z,V,C update
slt	0111	F = (A < B) ? 1 : 0
NOR	1100	F = ~ (A | B) (bitwise NOR); Z update, V,C=0

Truth Table:

The ALU operates by first modifying B based on F[2], which serves as kind of a sign bit. If F[2]=0, the ALU performs normal addition and logic using B. Otherwise, if F[2]=1, the ALU inverts B and adds 1 through carry-in, implementing subtraction using the 2's complement.

F[2:0]	F[2] selects BB	F[1:0] Y select	Operation	Output Y=
000	B	00	AND	A&B
001	B	01	OR	A |B
010	B	10	ADD	A+B
011	B	11	Sign-bit after add	{{N-1{1'b0}}, S[N-1]}
100	~B	00	AND ~B	A&~B
101	~B	01	OR ~B	A|~B

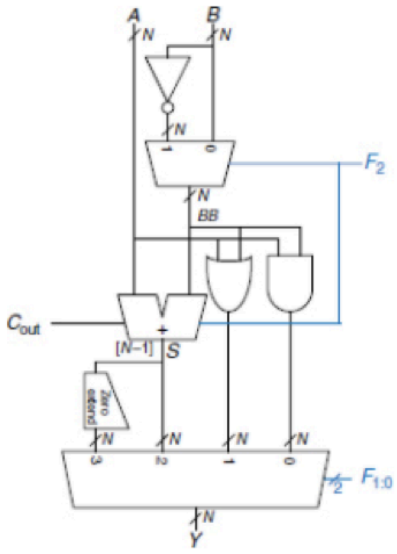
110	~B	10	SUB	A+~B +1 = A - B
111	~B	11	Sign-bit after sub	{{N-1{1'b0}}, S[N-1]}

Logic Equations:

Taking into consideration that the circuit uses MUX-selectors, it seems the equations are best expressed with piecewise functions:

- F2 selects BB:
  - BB $= \{B,\ F2\ =\ 0$
  - $= \{\sim B,\ F2\ =\ 1$
- Adder calculates S:
  - S =A + BB + F2
    - S $= \{A\ +\ B,\ F2\ =\ 0$
    - $= \{A\ -\ B,\ F2\ =\ 1$
- Carryout displays S[N] (S is internally treated as (N+1)-bit value)
  - Cout = S[N]
- Logic Paths:
  - and_out = { A & BB
  - or_out = { A | BB
- Final output selection using F[1:0]:
  - Y = {A & BB, F[1:0] = 00
  - = {A | BB, F[1:0] = 01
  - = S[N-1:0], F[1:0] = 10
  - = {{N-1{0}}, S[N-1]}, F[1:0] = 01
- Overflow occurs during signed add/sub when A, B have same sign but the sum doesn't (occurs only during arithmetic operations)
  - OV = (~(A[N-1] ^ BB[N-1])) & (A[N-1] ^ S[N-1])
- Zero flag set when result Y=0
  - ZF = ~ | Y → reduction NOR of Y's bits
-

Circuit/ Block Diagrams:



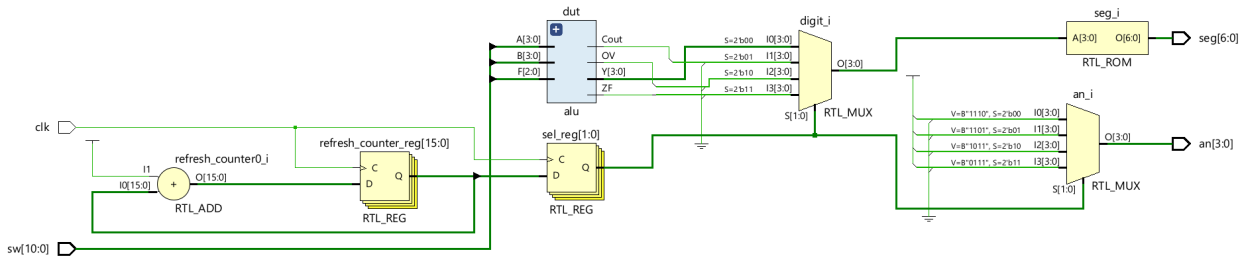
N-bit ALU circuit diagram from the lab manual

-> 2 n-bit inputs A, B
-> F2 chooses whether A subtracts (F2=1) or adds B (F2=0) → modifies B to BB
  -> when subtracting, F2 also adds a bit to the adder after selecting the inverted B from the multiplexer

-> perform A&BB, A|BB
-> Cout = carry out from A + BB
-> last multiplexer bits: (selected by F[1:0])

0	1	2	3
2'b00	2'b01	2'b10	2'b11
A & BB	A | BB	S = A +BB	Zero-extend/ sign bit

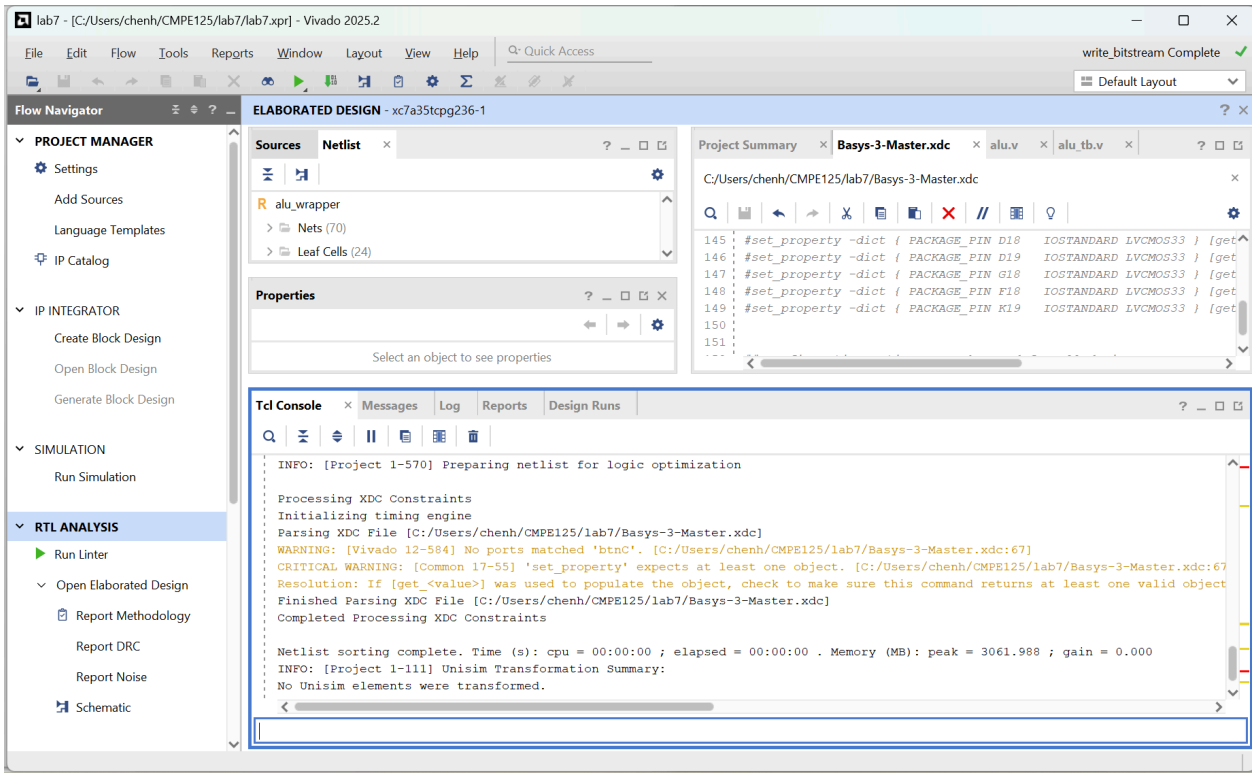
1. 2 N-bit inputs → A[N-1:0], B[N-1:0]
2. F2 chooses BB → (F2) ? ~B : B;
3. Ops happen in parallel
    a. Adder performs add/sub → S = A + BB + F2, Cout=S[N] (unsigned carry)
        i. * '+' means arithmetic addition, NOT 'OR' (this is not just a 1-bit adder)
    b. Bitwise AND → and_out = A & BB
    c. Bitwise OR → or_out = A | BB
4. F[1:0] selects final Y output
    a. 0 = 2'b00 outputs AND → Y = A & BB
    b. 1 = 2'b01 outputs OR → Y = A | BB
    c. 2 = 2'b10 outputs sum → Y = S[N-1:0]
    d. 3 = 2'b11 outputs *only* sign bit with zero-extend → Y = {{N-1{1'b0}}, S[N-1]}
5. Additional flag outputs:
    a. overflow flag V=1 when A & B have same sign but produce result with different sign → overflow = (A[N-1] == BB[N-1]) && (Y[N-1] != A[N-1])
    b. Zero flag Z=1 when output is zero → zero = ~|Y



Vivado-generated schematic

SCREENSHOTS

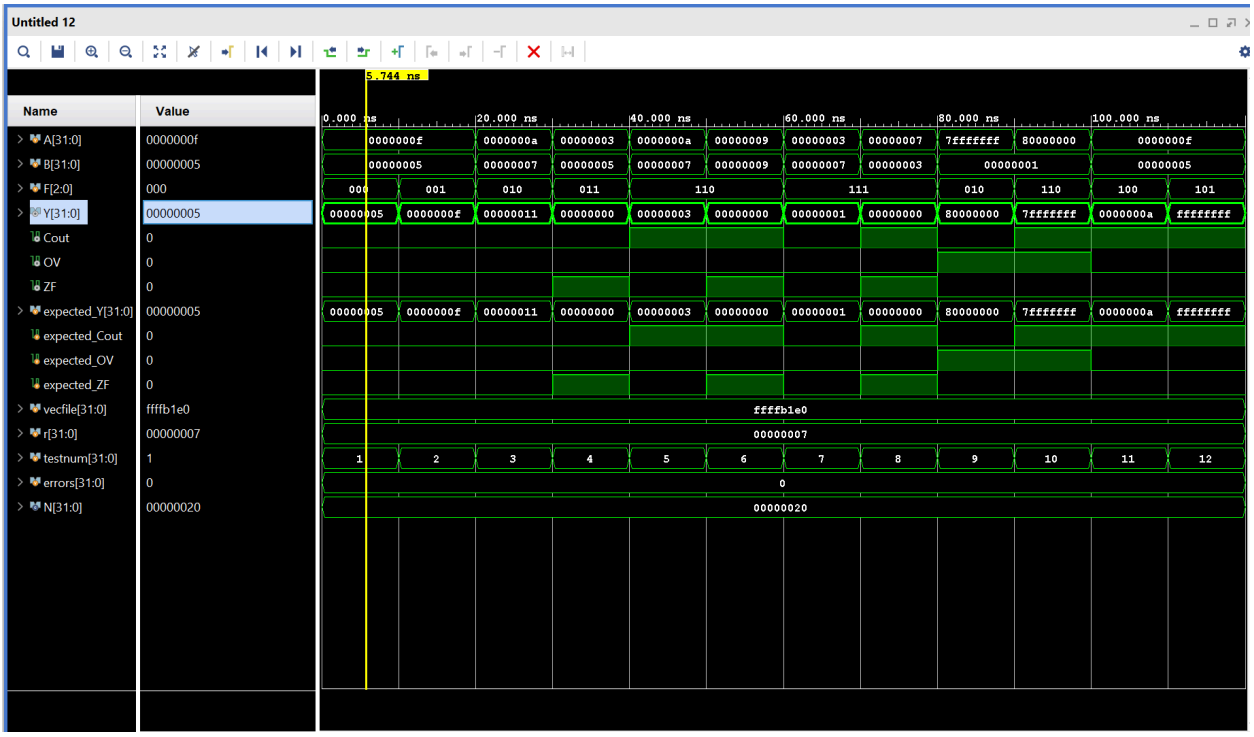
Compilation:



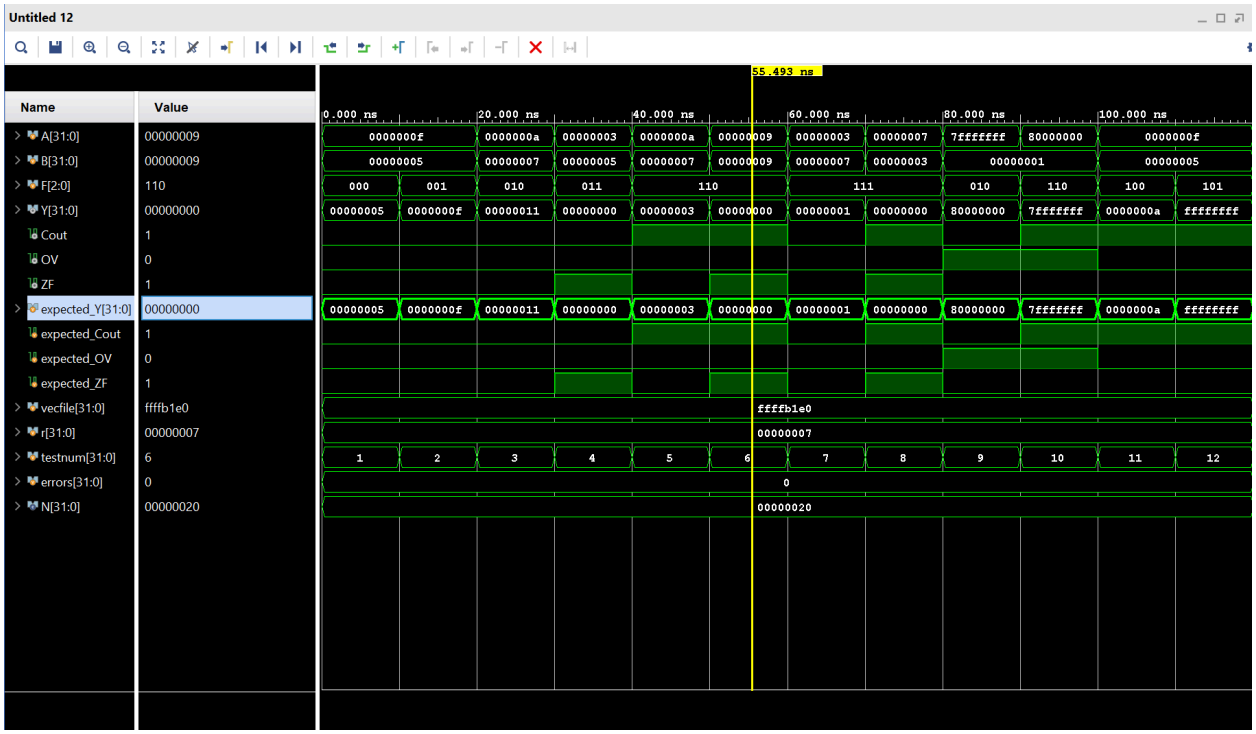
Vivado Window with "write_bitstream Complete" and "Netlist sorting complete" in Tcl consol

Simulation Waveforms:

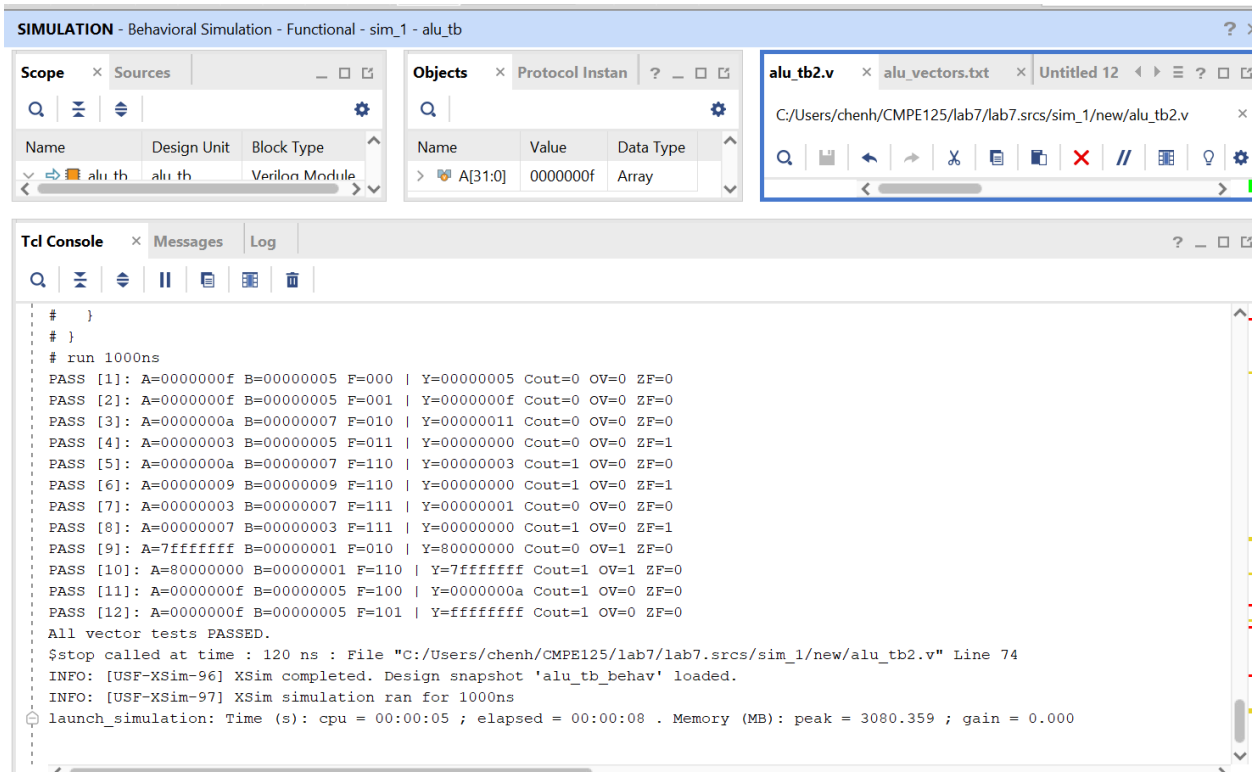
Waveform for all expected values match the instantiated modules' outputs.



A=0xf, B=0x5 → A&B=0x5



A=0x9, B=0x9 → A+~B = 0



Testbench console indicates all test vectors passed
(Cout always displays the carry out from the arithmetic operation, as do other arithmetic flags)

SUMMARY

Paragraph describing verification of implemented logic with the simulation results:
The objective of this lab was to design an Arithmetic Logic Unit (ALU) that models the one found in a MIPS (Microprocessor without Interlocked Pipelined Stages), which has a RISC-based architecture (32-bit registers and instructions). Characteristics of MIPS architecture also include using pipelining (arranging instructions to run in different stages simultaneously) and limiting memory access to specific instructions.
Due to limitations of the Basys3 board, 4-bit inputs and outputs will be used for A, B, and Y; however, the testbench should still implement the module with 32-bits.

VERILOG CODE

See attached files.

TROUBLESHOOTING

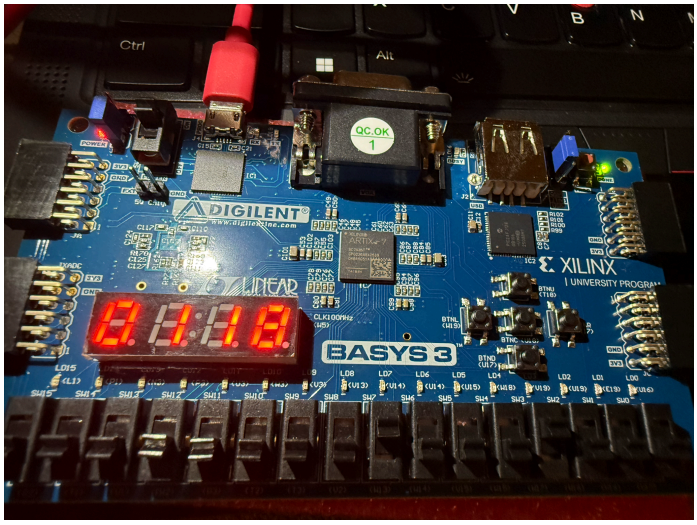
Several issues were encountered, mostly involving the interpretation of the ALU diagram and the implementation of the inputs and outputs. For the most part, the ALU logic was fine, as long as it was clarified that the flag signals would display the result of the arithmetic operation regardless of the F[2:0] select of the final Y. For instance, an issue arose from mismatches between

expected and actual Cout values for certain control inputs (F=100, 101), since it was initially assumed that Cout would only be valid when F[2:0]=2'b10, for arithmetic operations. However, this is not the case since the ALU design computes all datapaths in parallel, as shown in the diagram, and Cout and the other flags always reflect the result of the arithmetic operation regardless of the output. This issue was resolved by updating the testbench and clarifying this in the notes. There was also some confusion regarding signed number interpretation and which bit the sign flag SF would reflect. The ALU is supposed to interpret the MSB of Y, or Y[N-1] as the signed bit, and thus when Y=4'b1000, the signed bit =1 since the MSB=1.

FPGA BOARD OUTPUT

**4-bit inputs & outputs used for A, B, and Y

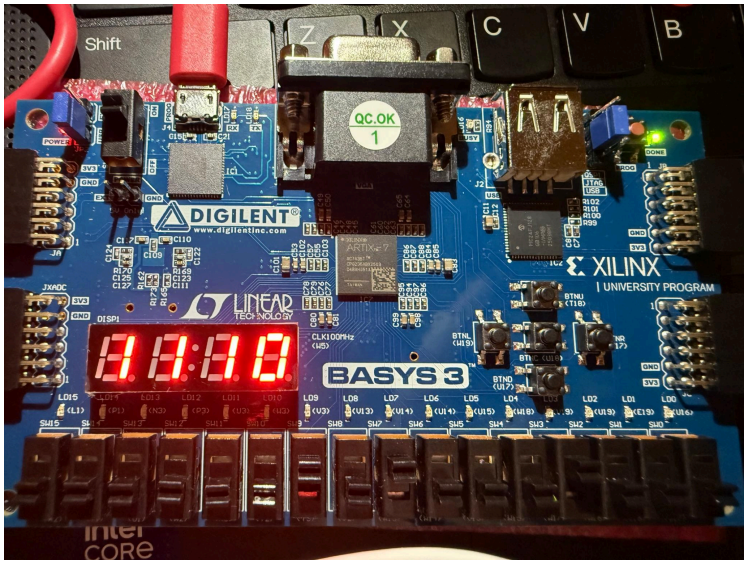
- An0: (rightmost anode) displays Y in hex
- An1: displays Cout
- An2: displays OVvvvvvvvvvvvvvvvvvvvvvvvvvvvvv
- An3: displays ZF (displays sign as result of arithmetic op regardless of F)
- Sw[7:4]: holds A in binary
- Sw[3:0]: holds B in binary
- Sw[10]: selects BB
- Sw[9:8]: selects final output Y



A=4'b1000, B=4'b1000, F2=0 (BB = B), F[1:0]= 2'b00 → A & B = 4'b1000

→ F = 8

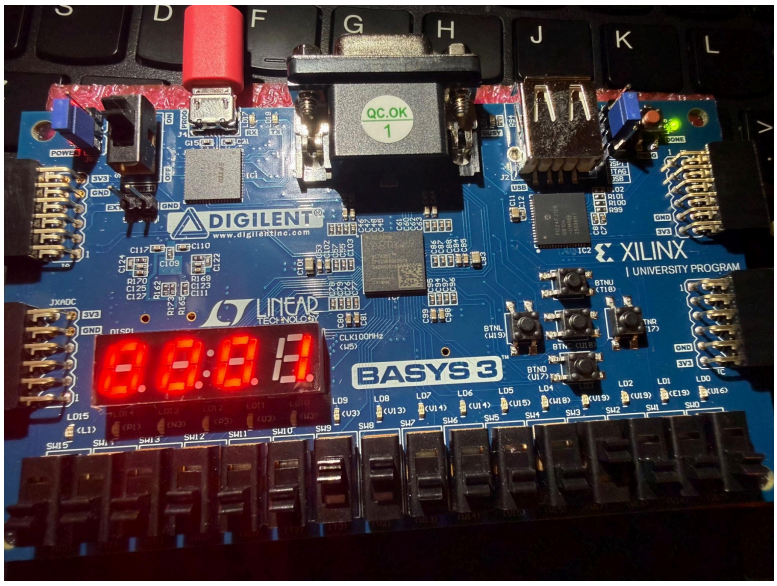
Arithmetic Flags for A + B = 16→ ZF=0, OV=1, Cout=1



A=4'b1000, B=4'b1000, F2=0 (BB = B), F[1:0]= 2'b10 → A + B = 5'b10000

→ F = 0

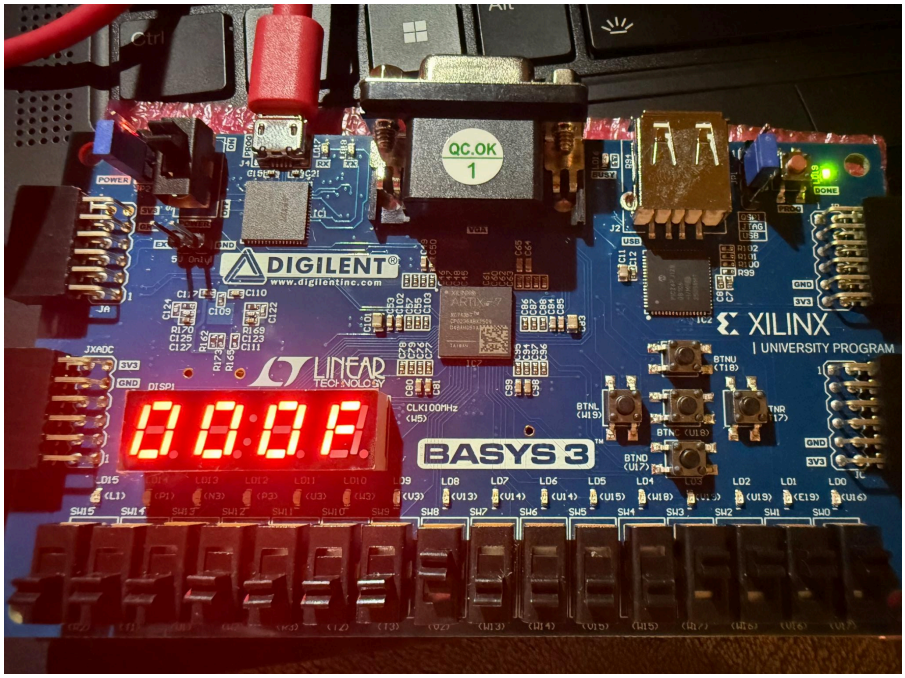
Arithmetic Flags for A + B = 16→ ZF=1, OV=1, Cout=1



A=4'b0000, B=4'b1000, F2=0 (BB = B), F[1:0]= 2'b11 → Sign bit of (A+B=4'b1000) →b4

→ F = 1

Arithmetic Flags for A + B = 8→ ZF=0, OV=0, Cout=0



A=4'b0000, B=4'b1111, F2=0 (BB = B), F[1:0]= 2'b01 → A | B = 4'b1111

→ F = F

Arithmetic Flags for A + B = 15→ ZF=0, OV=0, Cout=0